

XSkill 可插拔技能生产管线 重构设计

面向算法团队的接入、热配置与评测指南

核心命题：平台负责轨迹与 Skill 生命周期；算法内核只负责从标准输入生成标准产物。

把平台做稳定，把算法做可替换

这次重构的目标不是“多一个配置项”，而是建立可长期演进的算法边界。

XSkill = 稳定外壳 + 统一内核接口 + 版本化运行时 + 平台统一评价

接口稳定

1

算法只接收标准请求，只返回 SkillArtifact；不碰上传、用户统计和下发数据库。

包级注册

2

生产环境用 Python package entry point；开发环境保留 module:Class 快速调试。

无损切换

3

前端提交配置修订；新任务原子切换，在途任务继续固定旧版本，可一键回滚。

评价归平台

4

统一数据集、统一指标、统一裁决；算法可上报诊断，但不能决定自己是否胜出。

推荐起点：先把现有 AtomTask 管线封装成 builtin-atomtask 内核，行为不变；再开放第三方内核。

谁负责什么：边界必须一眼能看懂

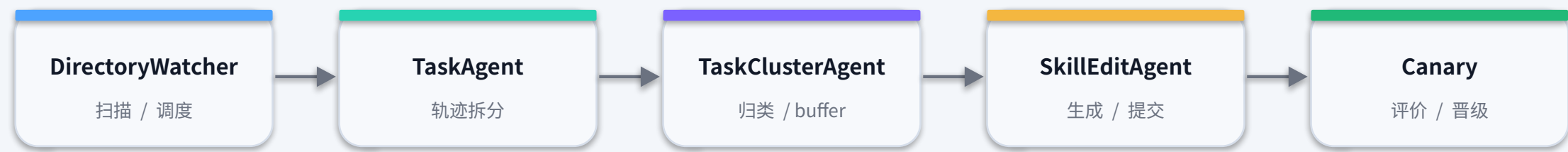
算法团队只对“如何生成 Skill”负责；平台对数据、用户和发布负责。



禁止跨界：内核不能直接更新用户统计、checkout Skill 分支或调用下发接口。

现有能力可复用，主要问题是“编排与实现绑死”

以下判断来自当前仓库 runner、TaskAgent、TaskClusterAgent、SkillEditAgent 与 canary 实现。



当前耦合点

直接构造类 Watcher 直接 import / 调用具体 Agent

共享内部状态 config、store、git、DB、目录结构同时暴露

没有版本固定 任务无法回答“由哪个内核 + 哪版配置生成”

替换粒度过细 改其中一段容易破坏跨阶段隐含约定

应保留的资产

AtomTask 数据模型 数据输入证据可追溯

候选 buffer 与 增量积累机制成熟值

main / staging 已经具备版本与灰度基础

UX / cost / model 统计 可直接成为统一评测数据

重构策略：把现有整条算法链先封装为一个默认内核，不立即重写算法逻辑。

从 OpenMines 学什么：统一基类 + 配置选择实现

OpenMines 用 BaseDispatcher 固定方法集合，用 JSON 选择算法；XSkill 需要在此基础上补齐生产治理。

OpenMines

BaseDispatcher + config

```
class BaseDispatcher:
    def give_init_order(self, truck, mine): ...
    def give_haul_order(self, truck, mine): ...
    def give_back_order(self, truck, mine): ...

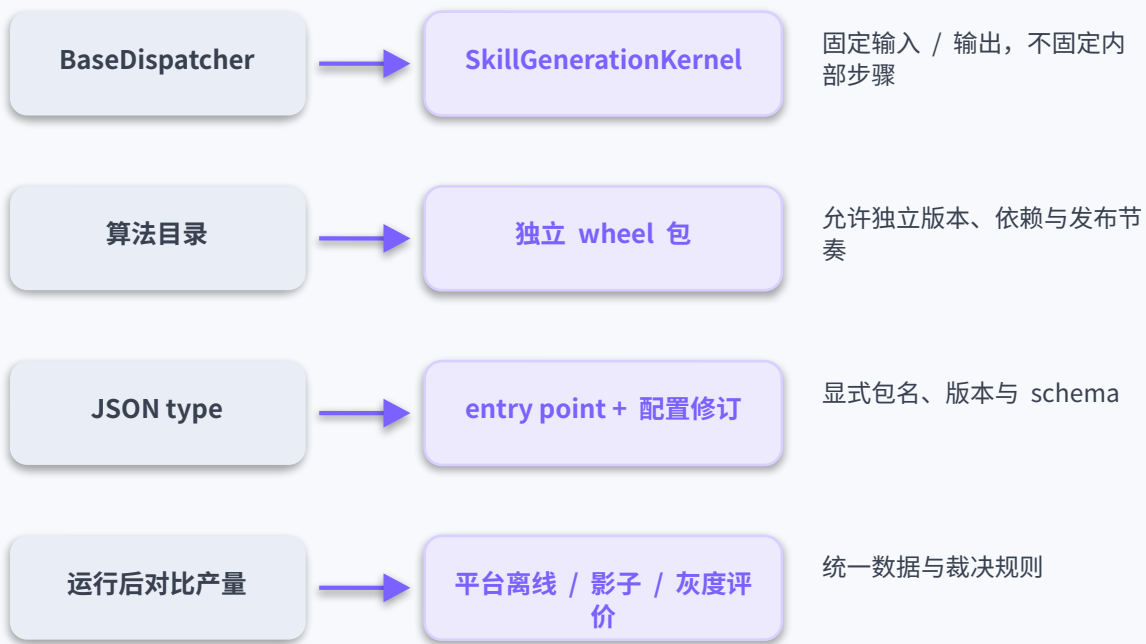
class NaiveDispatcher(BaseDispatcher):
    def give_init_order(self, truck, mine):
        return 0

# config.json
{"dispatcher": {"type": ["NaiveDispatcher"]}}
```

值得保留

- 基类定义最小接口
- 算法与框架分工清楚
- 配置决定运行实现

翻译到 XSkill



不照搬：生产环境不扫描目录猜类名；注册必须显式、可校验、可审计。

目标架构：控制、执行、数据与评价分层

对外保持一个 XSkill ； 对内把算法内核作为版本化插件运行。



替换的是算法，不能替换输入输出语义

统一契约让不同管线可以比较、重放、回滚，也让前端知道如何展示配置与结果。



统一抽象接口：算法团队只实现四个入口

接口要小、数据结构要强；不要把现有 TaskAgent / ClusterAgent 类直接变成公共 API。

```
xskill-kernel-sdk / contract.py

class SkillGenerationKernel(ABC):
    @classmethod
    def manifest(cls) -> KernelManifest: ...

    @classmethod
    def config_schema(cls) -> Mapping[str, Any]: ...

    def validate_config(self) -> None: ...

    def run(
        self,
        request: KernelRunRequest,
        services: KernelServices,
    ) -> KernelRunResult: ...
```

为什么只暴露 run ?

允许算法自由组织阶段

有的内核是三段 Agent，有的可能是端到端模型或工作流引擎。

减少跨版本锁定

平台不依赖 AtomTask、candidate buffer 等某一种内部实现。

评价以产物为中心

统一比较最终 Skill、lineage、成本和时延，而不是比较内部日志格式。

KernelServices 只给受控能力：读轨迹、调用模型、写临时产物、发事件。

推荐交付方式：独立 Python 包 + entry point

脚本路径适合本地调试；生产环境必须可版本化、可安装、可校验。

包目录

```
xskill-kernel-team-a/  
├── pyproject.toml  
├── README.md  
├── team_a_kernel/  
│   ├── kernel.py      ← SDK 适配层  
│   └── config.py      ← 配置模型 /  
schema  
├── algo_core/         ← 算法包  
├── resources/  
└── tests/  
    ├── test_contract.py  
    └── fixtures/
```

kernel.py 薄；核心算法仍由算法团队原包维护。

pyproject.toml

```
[project]  
name = "xskill-kernel-team-a"  
version = "1.3.0"  
dependencies = [  
    "xskill-kernel-sdk>=1,<2",  
    "team-a-algo==4.2.1",  
]  
  
[project.entry-points."xskill.skillgen_kernels"]  
team-a = "team_a_kernel.kernel:TeamAKernel"
```

两种加载模式

开发模式

module:Class；本地快速修改和调试。

生产模式

package + version + entry point；安装后进入注册表。

“热更换”定义：服务不停，新任务切换，在途任务不变

不要在同一 Python 进程里 reload 已运行的模块；依赖、缓存和全局状态都可能失控。



切换失败时只回滚 active revision 指针；不覆盖旧修订，不修改在途任务记录。

前端不是直接改 YAML ，而是提交不可变的配置修订

配置修订 = 内核身份 + 参数 + 发布策略；后端完成校验、预热和激活。

技能生产内核

内核

team-a

版本

1.3.0

配置修订

rev 42 （草稿）

发布方式

Shadow → Canary 20%

内核参数（由 config_schema 自动生成表单）

```
{"min_cluster_size": 3, "batch_size": 8}
```

校验并保存

后端事务

1

POST /kernel-configs

生成 rev 42 ， 状态 DRAFT

2

POST /kernel-configs/42/validate

schema + SDK + capability

3

POST /kernel-configs/42/prewarm

安装包、构造实例、smoke

4

POST /kernel-configs/42/activate

CAS 更新 active_revision

5

POST /kernel-configs/41/activate

回滚仍是一次激活

密钥只保存引用（secret_ref），不能进入算法配置正文。

评价必须与内核解耦：同题、同预算、同裁决

内核可以报告内部诊断；最终质量判断由 XSkill Evaluator 对标准产物执行。



统一评分卡



裁决顺序：先过硬门禁 → 再比较质量 → 质量相近时选成本更低者；不能只看一个加权总分。

算法团队交付一个新内核，只需要 5 步

下面从“已有算法包”出发，演示如何接入 XSkill，而不是要求算法团队重写业务逻辑。



Demo 项目：核心算法与 XSkill 适配层分开

PPT 同目录附带可运行代码：docs/ppt/xskill_kernel_demo。

xskill_kernel_demo/

```
|—— xskill_kernel_sdk.py    # 稳定接口（演示版）
|—— config.yaml            # 选择内核 + 修订
|—— pyproject.toml         # entry point
|—— run_demo.py            # 本地宿主
|—— demo_skillgen/
|   |—— algo_core.py        # 算法团队已有包
|   |—— kernel.py          # 薄适配层
```

运行：python3.11 run_demo.py

一次调用发生什么

标准请求

KernelRunRequest

适配层

DemoAtomTaskKernel

算法包

Splitter → Clusterer → Writer

标准结果

KernelRunResult

算法包完全不知道上传、用户统计、Skill 下发在哪里。

算法内核怎么写：继承基类，再调用自己的算法包

kernel.py 只做协议适配，不把算法逻辑搬进 XSkill。

demo_skillgen/kernel.py · 1/2

```
class DemoAtomTaskKernel(SkillGenerationKernel):
    def __init__(self, config):
        self.config = dict(config)
        self.splitter = SimpleSplitter()
        self.clusterer = KeywordClusterer()
        self.writer = MarkdownSkillWriter()

    @classmethod
    def manifest(cls):
        return KernelManifest(
            name="demo-atomtask",
            version="1.0.0",
            capabilities=("batch", "lineage"),
        )

    @classmethod
    def config_schema(cls):
        return DEMO_CONFIG_SCHEMA
```

demo_skillgen/kernel.py · 2/2

```
def run(self, request, services):
    atoms = []
    for traj in request.trajectories:
        text = services.read_text(traj.path)
        atoms += self.splitter.split(
            traj.trajectory_id, text
        )

    groups = self.clusterer.cluster(
        atoms,
        min_cluster_size=
            self.config["min_cluster_size"],
    )
    artifacts = [
        SkillArtifact(
            name=name,
            content=self.writer.write(name, members),
            source_trajectory_ids=source_ids(members),
        )
        for name, members in groups.items()
    ]
    return KernelRunResult(
        artifacts=artifacts,
        metrics={"atom_count": len(atoms)},
        lineage=build_lineage(artifacts),
    )
```

注意：调用模型、读取轨迹、写临时产物都走 services；不要 import xskill.pipeline.runner。

配置决定选谁，包元数据决定怎么找到它

开发时用 `class_path`；生产时用 `package + version + entry point`。

config.yaml

```
skill_generation:
  active:
    class_path: demo_skillgen.kernel:DemoAtomTaskKernel
    package: xskill-kernel-demo-atomtask
    version: 1.0.0
  config_revision: 42
  config:
    min_cluster_size: 1
  rollout:
    mode: shadow
    shadow_kernel: builtin-atomtask
```

pyproject.toml

```
[project]
name = "xskill-kernel-demo-atomtask"
version = "1.0.0"

[project.entry-points."xskill.skillgen_kernels"]
demo-atomtask =
    "demo_skillgen.kernel:DemoAtomTaskKernel"
```

smoke output

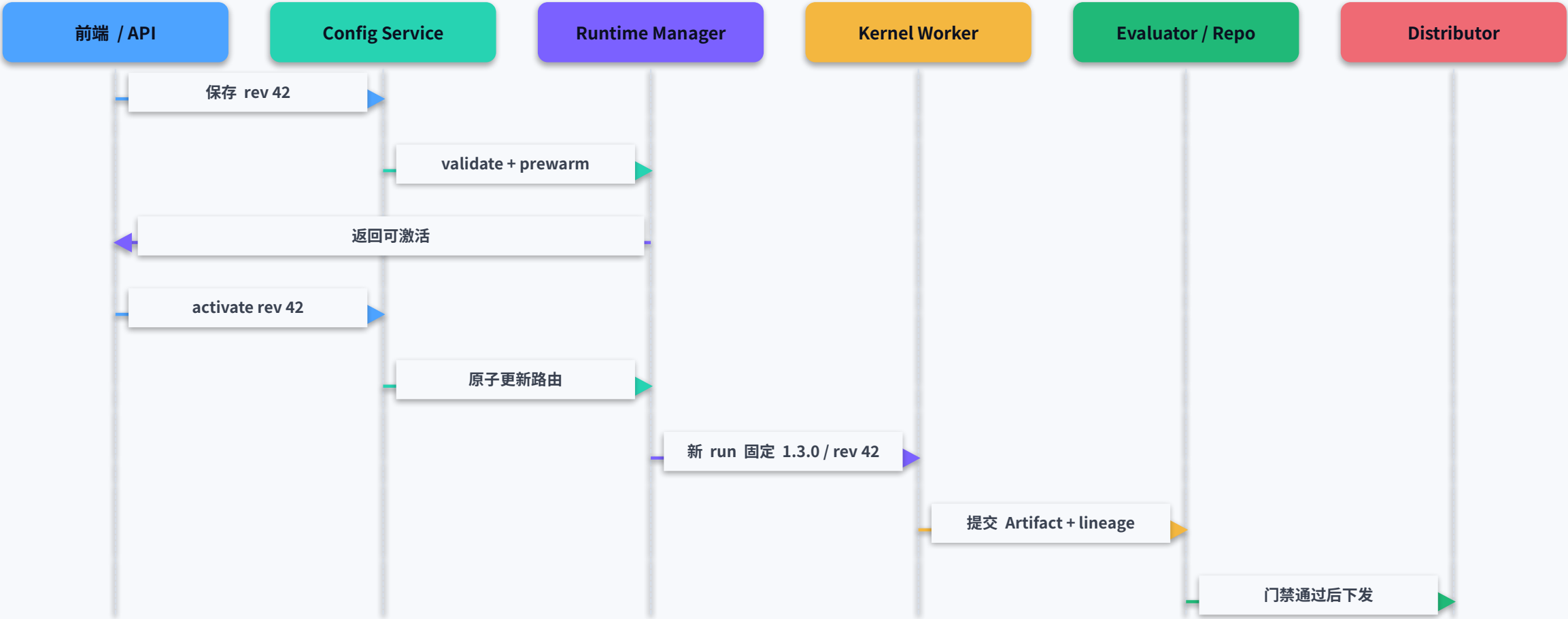
```
$ python3.11 run_demo.py
manifest: demo-atomtask@1.0.0
events: atoms.ready count=2
metrics: atom_count=2, skill_count=1
skills: ['修复']
lineage: {'修复': ['traj-demo']}
```

切换到新算法

只新增一个配置修订；旧配置仍保留，可重放、可回滚。

从前端切换到 Skill 下发：一条可审计的链路

每个关键动作都有状态、有版本、有回滚点。



状态真源：配置修订表 + run 记录 + Artifact 元数据；不要只依赖进程内对象。

先建立边界，再开放生态：四个阶段逐步迁移

每个阶段都能独立验收，不要求一次性把当前管线重写完。



今天需要拍板的 4 个决定

1

公共接口以整条管线 run 为粒度

2

生产插件只接受 wheel + entry point

3

热切换只影响新任务，所有 run 固定版本

4

评价与发布由平台统一控制

算法团队只需记住三件事

01 继承统一基类

实现 `manifest / config_schema / validate_config / run`。

02 调用自己的算法包

XSkill 适配层要薄，不依赖平台内部模块。

03 交付可评价产物

返回 `SkillArtifact`、`lineage` 和诊断；平台负责是否发布。

接入检查

- ✓ 包有明确版本
- ✓ 配置可校验
- ✓ 不直接访问平台 DB
- ✓ 结果有 lineage
- ✓ 合同测试可重复
- ✓ 支持失败与超时
- ✓ 通过影子后再灰度

下一步：用 `builtin-atomtask` 完成 P0 / P1 原型，拿真实轨迹做等价性回归。

Q & A